

RESEARCH ARTICLE OPEN ACCESS

Methods for Setting Device Specifications for Analog In-Memory Computing Inference

Zhenyu Wu¹ | Xin Su² | Malte J. Rasch³ | Ning Li^{2,4} 

¹Department of Computer Science and Engineering, Lehigh University, Bethlehem, Pennsylvania, USA | ²Department of Electrical Engineering, Pennsylvania State University, University Park, Pennsylvania, USA | ³Sony AI, Sony Europe B.V., Schlieren, Switzerland | ⁴Material Research Institute, Pennsylvania State University, University Park, Pennsylvania, USA

Correspondence: Ning Li (nbl5288@psu.edu)

Received: 29 October 2025 | **Revised:** 29 January 2026 | **Accepted:** 30 January 2026

Keywords: device benchmarking | in-memory computing | non-volatile memory | phase change memory

ABSTRACT

Analog in-memory computing (AIMC) based on non-volatile memories (NVMs) offers a transformative pathway toward ultra-energy-efficient deep learning, enabling high-speed, low-power inference with scalable performance. Recent progress has demonstrated floating-point-level accuracy through hardware-aware training, highlighting AIMC's potential to rival conventional digital architectures. As NVM technologies for AIMC continue to advance, a systematic framework for defining device specification targets is critical for balancing inference accuracy and fabrication cost. Key device parameters, such as memory window, read and program noise, and conductance drift, directly influence inference fidelity across different time scales while simultaneously affecting manufacturability. Their strong interdependence complicates optimization, as tuning one parameter can relax or tighten constraints on others. To address this challenge, we propose a comprehensive methodology for mapping the multidimensional device specification space required to achieve floating-point-equivalent accuracy, using phase-change memory (PCM) as a representative platform. Cross-model evaluations on convolutional neural networks, recurrent neural networks, and transformer architectures reveal that device characteristics must be co-optimized within this coupled landscape. The resulting analysis identifies families of specification sets that deliver comparable accuracy, enabling flexible, cost-effective hardware design and paving the way for scalable, commercially viable AIMC deployment.

1 | Introduction

As AI development advances into the era of large-scale models, the traditional von Neumann architecture is increasingly limited by the memory wall, constraining both energy efficiency and latency in AI computing. Analog in-memory computing (AIMC), which performs computation directly within memory arrays, offers a potential path to overcoming this bottleneck. By reducing data movement, AIMC can deliver significant improvements in energy efficiency and throughput, making it attractive for both resource-constrained edge devices and high-performance data centers [1, 2]. Recently, as a result of the rapid development of research and development in this area, there is a trend of accelerated growth of the memory industry for computing applications [3]. Floating point accuracies have been achieved using the hardware-aware

training methods [4–12] in both simulation [9–13] and chip demonstrations [6, 7]. However, the analog memory device non-idealities, such as noise, drift, limited memory window, etc., have hindered wide deployment of AIMC. It is highly desirable to mitigate the challenges of managing analog device non-idealities by defining specification targets that are both achievable and capable of ensuring high accuracy. We developed a method to do that in this work, by finding all the combinations of relevant device characteristics that are able to achieve the floating point accuracies. This allows the manufacturer to pick the device specification targets that are easiest to achieve to reduce the barrier of device development and the cost of manufacturing.

Prior studies on the impact of memory characteristics on AI tasks mostly focus on individual non-ideality [6, 10, 13], such as the

This is an open access article under the terms of the [Creative Commons Attribution](https://creativecommons.org/licenses/by/4.0/) License, which permits use, distribution and reproduction in any medium, provided the original work is properly cited.

© 2026 The Author(s). *Advanced Intelligent Systems* published by Wiley-VCH GmbH.

effect of memory window or conductance drift on the inference accuracy. However, the accuracy of AI tasks is affected by the combined influence of multiple non-idealities, which are interdependent with one another. The method of investigating one parameter at a time has intrinsic limitations. The collective impact of various device characteristics is complex and not easily predictable. In addition, the impact of these device characteristics is usually time-dependent. What may appear optimal for short periods after programming does not necessarily hold for later times. For instance, mitigating drift or noise alone may fail to preserve model accuracy over extended durations if the memory window of this type of device is limited. Ensuring an optimal and reasonable device design requires more holistic design strategies. In this paper, we explore the collective impact of all relevant factors, including memory window, conductance drift, read noise, and inference time, on the inference performance of Deep Neural Networks (DNNs). We consider how these factors evolve and interrelate over time under all possible configurations. It should be noted that the device specification to maintain good accuracy is not unique. It is difficult to assert that any single device configuration is unequivocally better than another without considering the target application's requirements and the constraints on device characteristics. However, some configurations might be more preferred due to various reasons such as design considerations or material and fabrication-related reasons. Our method presents all possible combinations of device characteristics that achieve floating-point accuracy while allowing flexibility in selecting from different device manufacturers.

We use phase change memory (PCM) as an example to illustrate the method of finding all valid device specifications. Among the various devices suitable for AIMC, PCM has received much attention because of its favorable attributes [3, 14–17], including analog conductance programming, appropriate resistance levels, a robust memory window, multilevel resistance states, high endurance, relatively mature integration technology compared to other emerging memory devices, and the ability to perform large-scale matrix vector multiplications (MVMs). These advantages position PCM as an excellent device candidate for AIMC, particularly for inference applications, which account for more than 80% of the computational resources used in AI workflows [18]. Similar to any other analog NVMs [19–21], PCM has its intrinsic non-idealities, such as read noise [12, 22, 23], device-to-device

variations [24], conductance drift over time after programming [13, 25, 26], limited memory window [17, 27], which can adversely affect DNN inference accuracy. These are very typical for analog NVM devices.

Our study starts with training three representative DNNs—ResNet [28] (a convolutional neural network (CNN)-based architecture), LSTM [29] (an recurrent neural network (RNN)-based architecture), and BERT [30] (a transformer-based architecture)—to iso-accuracy through noise injection and related techniques as described in Ref. [12]. We then program these HWA DNN weights into simulated PCM devices. The PCM device drift and noise are modeled with the PCM noise model [31], implemented via the IBM Analog Hardware Acceleration Kit (AIHWKIT) [32–34]. By systematically sweeping a multi-dimensional parameter space encompassing memory window, drift, and noise across multiple inference timescales, we generate comprehensive evaluations and complete device specification ranges capable of sustained iso-accuracy. These results can be used as a guide to the selection of memory window, drift, and noise parameters for a given inference time and performance target. Rather than relying on trial-and-error or overly simplified assumptions, our approach offers a guideline for identifying a viable range of device configurations to support the development of more robust or more cost-effective designs in analog in-memory computing.

2 | General Approach

The general approach of this work is illustrated in Figure 1. First of all, we begin with training floating point (FP) DNN models to achieve state-of-the-art accuracy by either using a pre-trained model or training the model from scratch, depending on the availability. These FP models are then mapped to conductance values of NVM devices in the AIMC crossbar arrays using the AIHWKIT [32–34] simulation framework. To mitigate the impact of NVM non-idealities and obtain robust weight values for AIMC, we apply the hardware-aware (HWA) training procedure [5, 12]. The HWA method incorporates device imperfections, such as noise injection detailed in the Methods section, during the training process. Here we use the PCM noise model [6, 12, 31] as an example. This training method enables the development of models that achieve or approximate iso-accuracy (within 1% of the floating-point reference) on AIMC systems. Such models can then serve

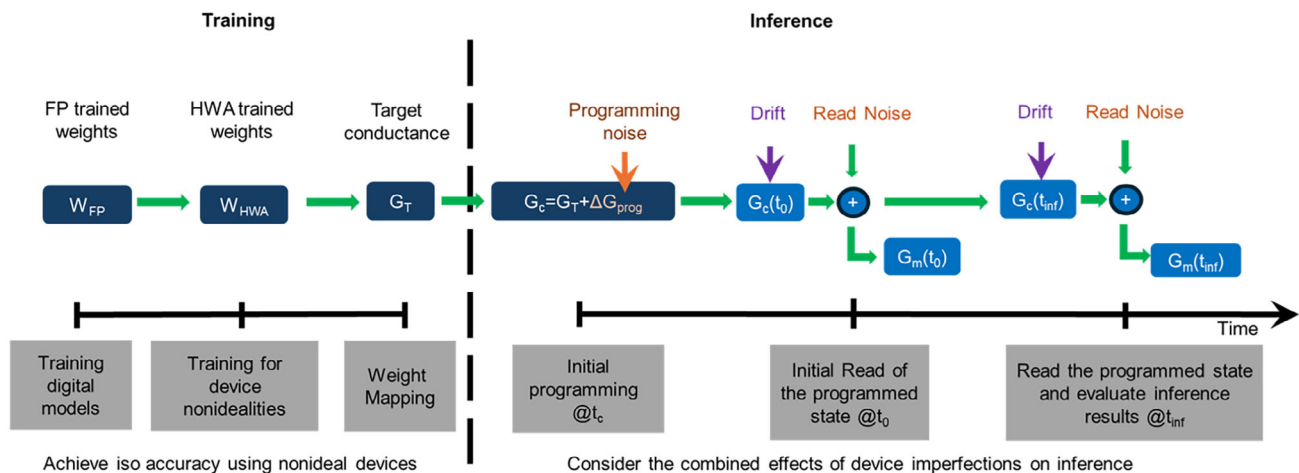


FIGURE 1 | Block diagram of the general approach used in this work.

as a foundation for investigating the impact of PCM non-idealities on inference tasks.

After obtaining the robust weight values in conductance for AIMC to achieve iso accuracy, we conduct the inference study using the AIHWKIT, which provides a state-of-the-art statistical model for NVM arrays, including PCMs. This statistical model is used when performing inference to simulate the various sources of non-idealities that are present in a real hardware [31]. The inference model is calibrated based on extensive measurements performed on an array containing 1 million PCM devices fabricated at IBM [6]. The specifications of the PCM devices used for calibration are summarized in Table S1. During the inference study, the PCM noises and drift are incorporated using statistical models across the NVM array. The DNN and datasets we used include Image classification on CIFAR-10 [35] for ResNet-32, language processing tasks using SQuAD v1.1 [36] and GLUE (MRPC) [37] datasets for the BERT model, and word-level prediction using the Penn Treebank (PTB) [38] dataset for the LSTM.

The device characteristics used in the study are defined as follows. The memory window is defined as the linear programming range, $g_{\max} - g_{\min}$, where $g_{\min}$ denotes the minimal conductance (i.e., the value that the logical zero of the weights will be mapped to) and $g_{\max}$ denotes the maximal conductance (i.e., the value that the absolute max of the weights will be mapped to). The noise scale and drift scale are defined as scaling factors compared to the AIHWKIT default device model, which was obtained from the statistical data from PCM devices [6, 12, 31]. The noise scale is applied to both the programming noise and read noise, since the programming noise is fundamentally limited by the read noise in the case of iterative programming [6]. Details of the PCM statistical device model are provided in the Methods section, and the corresponding base parameters are summarized in Table S2. Following prior work [12], we define normalized accuracy $\mathcal{A}_*^t$ as

$$\mathcal{A}_*^t = 1 - \frac{\epsilon_t^{\text{inference}} - \epsilon^{\text{FP}}}{\epsilon^{\text{chance}} - \epsilon^{\text{FP}}} \quad (1)$$

where, ϵ denotes the error rate (i.e., $1 - \text{accuracy}$), $\epsilon_t^{\text{inference}}$ is the AIMC test error at inference time t of the HWA DNNs, ϵ^{FP} is the test error of the FP DNNs, and $\epsilon^{\text{chance}} = 1 - \frac{1}{\text{Number of classes}}$

is the test error from random guessing. By this definition, $\mathcal{A}_*^t = 1$ corresponds to FP-level performance and $\mathcal{A}_*^t = 0$ corresponds to chance-level performance. Through re-scaling, the AIMC test error $\epsilon_t^{\text{inference}}$ at time t by the difference between the original FP test error and the test error from random guessing in Equation (1), we can quantify the accuracy drop across different device parameters and different inference time. We define the iso-accuracy criteria as $\mathcal{A}_*^t \geq 99\%$ at time t (1% accuracy degradation with respect to FP model).

For all the DNNs studied here, HWA training yields AIMC models that achieve iso-accuracy using standard PCM noise [6, 12, 31], consistent with the results of prior works [12]. We then use these HWA models in systematic inference experiments, where we analyze how different hardware parameters, including memory window, noise (both programming noise and read noise), drift, and inference time, affect the overall inference accuracy and robustness of each model. In the following sections, we first discuss the results of BERT to show the trend for large language models. We then focus on LSTM and ResNet for more detailed comparisons of various combinations of device characteristics.

3 | Results for BERT

After applying the training method detailed in General Approach section and Method section, we achieve iso-accuracy for both the MRPC dataset and SQuAD dataset. The hyperparameters used for hardware-aware training are detailed in the Method section. Figure 2a shows the inference accuracy as a function of the device noise and the memory window at 1 s after weight programming. The red plane marks a 1% EM-drop threshold which is recognized as the tolerance limit for performance degradation. The yellow region denotes the feasible device specifications. It is clear that there are different combinations of the noise and memory window that can meet the requirement of the iso-accuracy criteria.

Figure 2b adds the drift scale as an influence factor and shows the inference accuracy as a function of the drift time, drift scale, and noise scale. Semi-transparent planes correspond to the drift scales of 0.01, 0.1, 0.5, and 1.0. The larger drift scales accelerate the accuracy decay over time. In addition, the degradation is stronger at higher noise scales. It is clear that these factors need to be considered

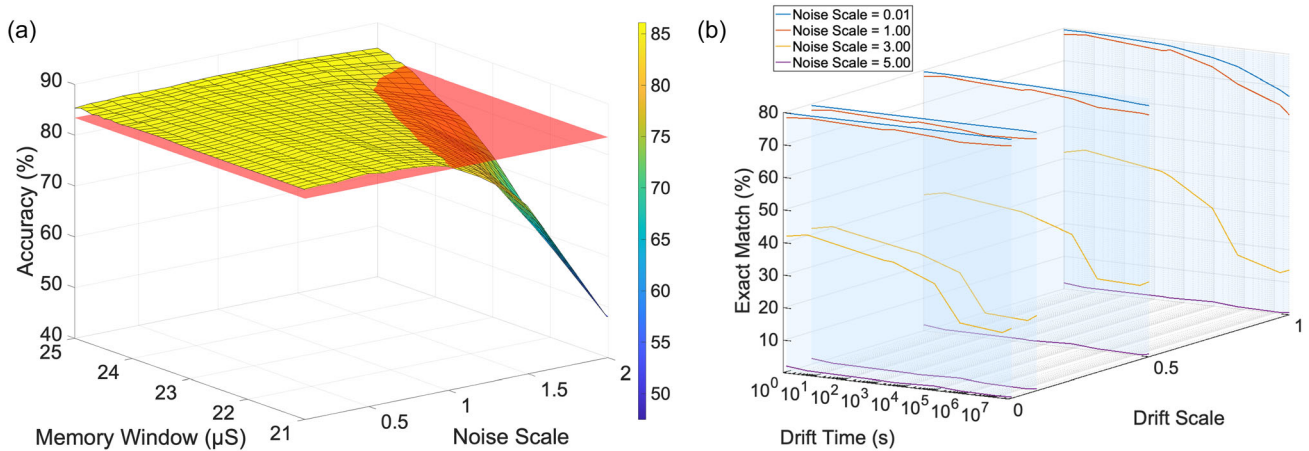


FIGURE 2 | Accuracy of BERT-base under combined device non-idealities. (a) MRPC inference accuracy versus noise scale and memory window at 1 s after programming. (b) SQuAD v1 exact match over drift time for multiple noise scales. Semi-transparent planes correspond to drift scales 0.01, 0.1, 0.5, and 1.0.

together to determine what the best configuration is to meet the accuracy criteria at certain time after programming. And there are more than one options to achieve the same target. These results show a multidimensional parameter space in which many device characteristics jointly determine the system-level performance.

4 | Results for LSTM

After applying the training method detailed in the General Approach section and Method section, including both FP training and HWA training, the HWA LSTM model, denoted as LSTM_{HWA}, achieves iso-accuracy for PCM based AIMC model [6, 12, 31]. After obtaining the robust HWA-trained weight values and in conductance, we perform inference studies using AIHWKIT over five inference time $t \in \{1 \text{ s}, 1 \text{ h}, 1 \text{ day}, 1 \text{ week}, 1 \text{ year}\}$, while systematically scanning the parameter space of device characteristics. As described in the approach section, the device imperfections, including resistance drift, read noise, and memory window, were examined in the inference study. Following Equation (1), the LSTM_{HWA} achieves $\mathcal{A}_*^t \geq 99\%$ across all 5 evaluated inference time t (1 s, 1 h, 1 day, 1 week and 1 year) under the standard PCM noise model, with $\mathcal{A}_*^{1h} \approx 99.7\%$ (normalized accuracy at 1 h after programming) and

$\mathcal{A}_*^{1y} \approx 99.1\%$ (normalized accuracy at 1 year after programming). These results show that RNNs exhibit remarkable robustness against device non-idealities after HWA training compared to other DNNs. A detailed table summarizing all evaluated periods is provided in Table S4.

Figure 3 shows a multi-dimensional device parameter space in terms of memory window, noise scale, and conductance drift, across five different inference time t : 1 s (Figure 3a), 1 h (Figure 3b), 1 day (Figure 3c), 1 week (Figure 3d), and 1 year (Figure 3e). The surface represents the valid combinations of these parameters that achieve $\mathcal{A}_*^t \geq 99\%$ over the specified time period t . Each point on this surface corresponds to a specific device specification that achieves this accuracy threshold. Additionally, the space below the surface (enclosed by it) represents parameter combinations that satisfy the $<1\%$ accuracy degradation criterion. In the short-term scenario, the parameter space accommodating the target accuracy degradation is relatively broad. At 1 s and 1 h, the surfaces along the drift scale dimension are relatively flat and show declining slopes along the noise scale dimension as noise scale increases. For example, at a fixed noise scale, achieving the target iso-accuracy with different drift scales requires nearly the same memory window. This suggests the LSTM_{HWA} is robust against and insensitive to the

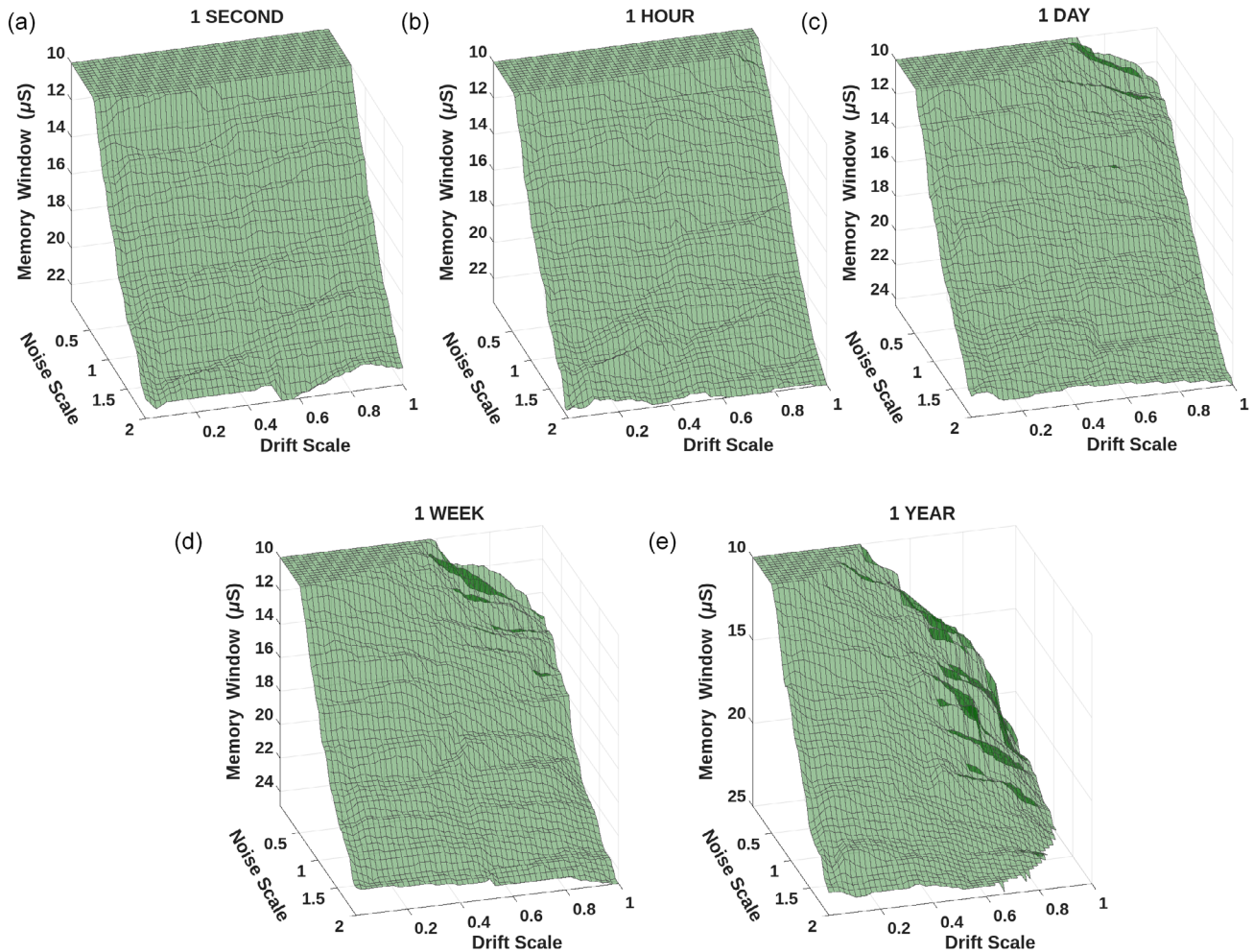


FIGURE 3 | 3D contour plot of PCM device specifications for LSTM_{HWA}. The surface of inference results maintained iso-accuracy $\mathcal{A}_*^t \geq 99\%$ (test perplexity of approximately 87.8) with combinations of memory window, noise scale (scale of both programming noise and PCM read noise) and drift scale across 5 different periods t : (a) 1 s, (b) 1 h, (c) 1 Day, (d) 1 Week, (e) 1 Year.

increase in the drift scale in short-term scenario, but the memory window needs to be enlarged to compensate large noise scale. As the time period extends from 1 day to 1 year, the feasible parameter space contracts, especially along the drift scale dimension where the surfaces progressively exhibit a downward trend as drift scale increases. Over longer durations, maintaining $<1\%$ accuracy degradation requires larger memory windows as both noise and drift scales increase. Consequently, the viable parameter space becomes more restricted. In addition, Figure S1 shows the feasible parameter space under iso-accuracy thresholds of $A_t^* \geq 99.5\%$ (Figure S1a) and $A_t^* \geq 98\%$ at 1 year (Figure S1b). Imposing a stricter accuracy requirement slightly reduces the feasible parameter region, whereas relaxing the requirement moderately enlarges it. However, the overall dependence on memory window, drift scale, and noise level remains consistent across all three thresholds. These results indicate that the same device-level trade-offs apply over a practical range of accuracy targets, allowing the parameter space to be interpreted in the context of different application requirements. The exact device specifications can be updated based on the accuracy requirement. To further guide hyperparameter selection and optimization, Figure S3

visualizes the gradient field, ∇A_t , of accuracy sensitivity across the normalized noise-memory window parameter space. The gradient direction indicates the accuracy improvement path, while the gradient magnitude (encoded by arrow length) shows parameter sensitivity. We analyze three time horizons (1 h: Figure S3a,d,g; 1 Week: Figure S3b,e,h; 1 Year: Figure S3c,f,i) under three drift scales (1.0, 0.5, 0.05). This figure shows that accuracy is most sensitive under long inference periods with large drift scale (1 Week and 1 Year at drift scale = 1.0), where the gradient are large and oriented toward increasing memory window. When the drift scale is reduced to 0.5 or 0.05, the largest gradients concentrate in the low noise, low memory window corner and still tend to point toward larger memory window. At moderate noise levels (below a normalized noise scale of about 0.5), the gradient component along the memory window direction weakens and the gradients increasingly point toward lower noise, with magnitudes also decreasing. In all settings, the high-memory, low-noise region exhibits very small gradients, suggesting that accuracy is largely insensitive in this part of the parameter space.

To provide a quantitative view of these parameter spaces, Figure 4 summarizes the noise scale tolerance across memory

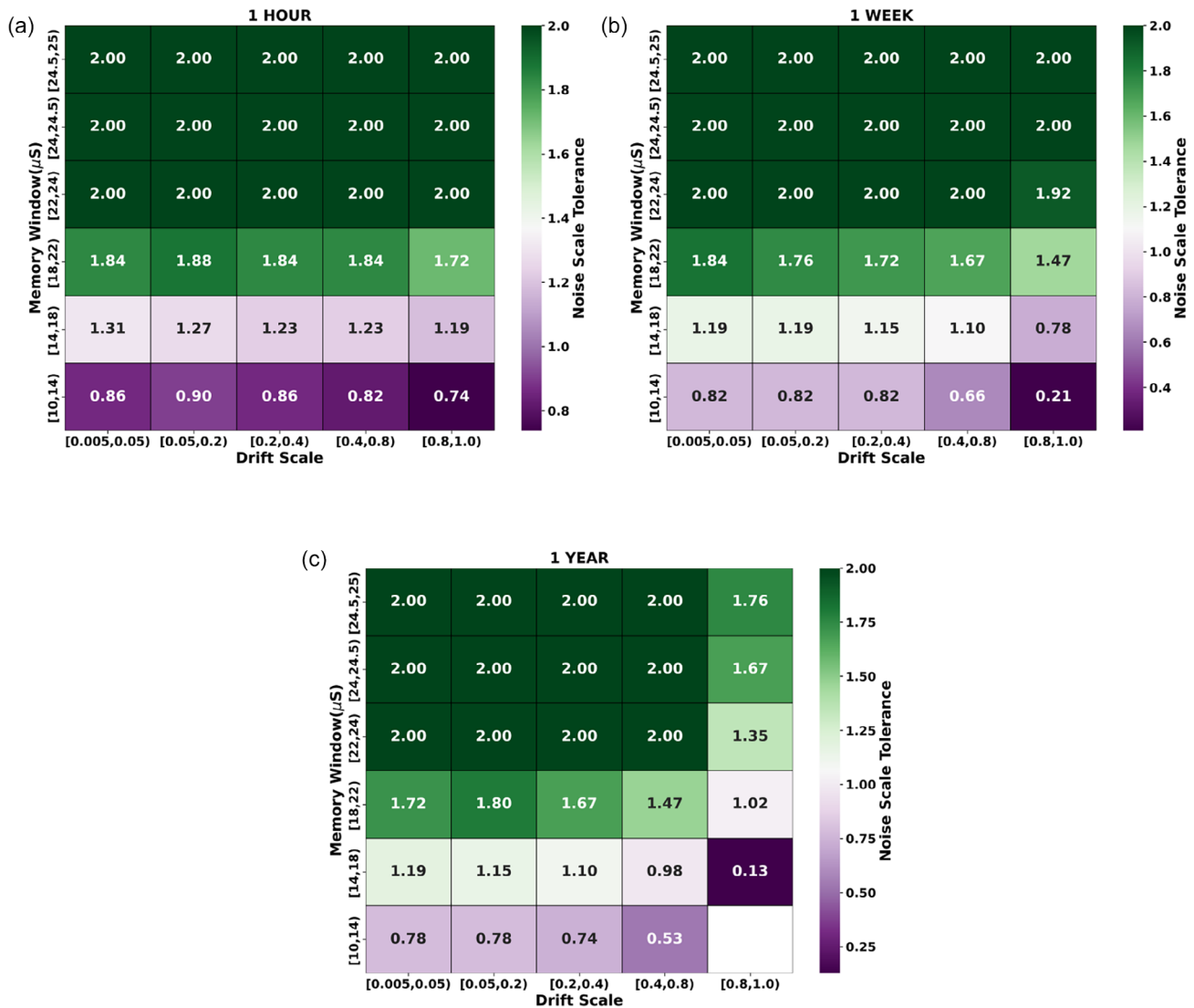


FIGURE 4 | PCM device specifications lookup table for LSTM. The number in the colored grids show the tolerable noise scales at various memory windows and resistance drift scales achieving $A_t^* \geq 99\%$ (test perplexity of ≈ 87.8) at the inference times of: (a) 1 h, (b) 1 Week, (c) 1 Year.

window and drift scale intervals for the 1 h (Figure 4a), 1 week (Figure 4b), and 1 year (Figure 4c) scenarios. Specifically, the volume of valid parameter combinations was projected into heatmaps, where each grid cell corresponds to a memory window range and a drift scale range. The annotated value of each cell represent the maximum noise scale that can be tolerated within that region while still satisfying the $<1\%$ accuracy degradation criterion. This figure can serve as a lookup table to determine feasible device specifications for a given accuracy target and retention time. For example, once a maximum allowable drift scale is specified, the corresponding grid row directly indicates the minimum required memory window and the maximum tolerable noise scale needed to maintain $<1\%$ accuracy degradation. As an example, to achieve iso-accuracy at 1 year under a relatively high-drift condition (drift scale ≥ 0.8), compensating for a noise scale of 1 requires a memory window of at least 18 μS . In this way, the figure provides a practical design reference supporting the selection of device specifications that satisfy system-level performance requirements.

Figure 5 shows the heatmap of the inference accuracy of LSTM_{HWA} under three drift scales (1.0, 0.5, and 0.05) at inference times of 1 h, 1 week, and 1 year. Device characteristic specifications that can sustain iso-accuracy, $\mathcal{A}_*^t \geq 99\%$, are highlighted in green. Results for 1 s and 1 day are provided in the Figure S4. Figure 5a–c show the inference result of the LSTM_{HWA} with drift scale = 1 at inference times of 1 h, 1 week, and 1 year, respectively. At shorter inference durations, there exists a broader set of acceptable device specifications that achieve the target accuracy. In addition, reducing the memory window leads to diminished noise tolerance in the LSTM_{HWA} model. For instance, to maintain iso-accuracy at $t = 1$ h, a large memory window (e.g., 24 μS) can tolerate noise scales up to 2.0, while a small memory window (e.g., 10 μS) supports only up to a noise scale of 0.1. Over extended inference durations, this trend persists, and the set of configurations achieving iso-accuracy further shrinks. At $t = 1$ year, feasible configurations are largely confined to low noise scales (e.g., <1.0) and large memory windows (e.g., $>24 \mu\text{S}$).

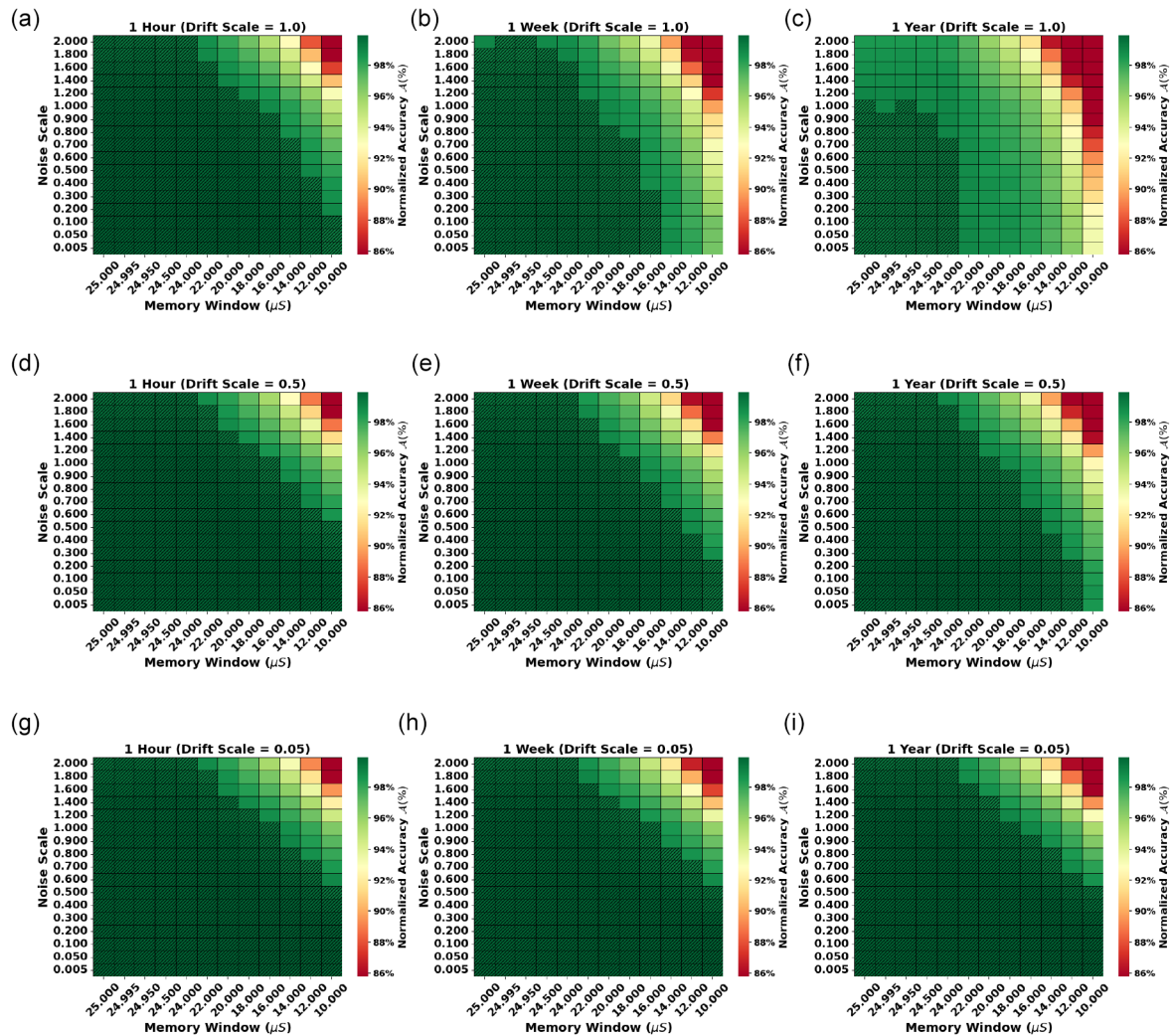


FIGURE 5 | 2D heatmap of inference accuracy for LSTM_{HWA} for PCM devices with various characteristics. Grids show the normalized accuracy achieved with the varying memory window and noise scale (scale of both programming noise and PCM read noise) across 3 different periods and 3 different drift scales: (a) 1 h (drift scale = 1.0), (b) 1 Week (drift scale = 1.0), (c) 1 Year (drift scale = 1.0), (d) 1 h (drift scale = 0.5), (e) 1 Week (drift scale = 0.5), (f) 1 Year (drift scale = 0.5), (g) 1 h (drift scale = 0.05), (h) 1 Week (drift scale = 0.05), and (i) 1 Year (drift scale = 0.05). Grids achieving iso-accuracy $\mathcal{A}_*^t \geq 99\%$ are highlighted (test perplexity of approximately 87.8).

After reducing drift scale to 0.5, we re-evaluated the accuracy of LSTM_{HWA} at 3 inference time t : 1 h (Figure 5d), 1 week (Figure 5e), and 1 year (Figure 5f). Reducing the drift scale relaxes the requirements of device characteristic specifications to achieve iso-accuracy, especially in the long-term scenario. At 1 h, the change in device specification requirements is marginal, with no significant difference from the case of drift scale = 1. On the other hand, at 1 year, the set of feasible device specifications expands considerably, toward larger noise scales and smaller memory window values. At $t=1$ year, LSTM_{HWA} tolerates noise scales up to 2.0 with large memory windows (e.g., $>24 \mu\text{S}$), and can also maintain iso-accuracy under tighter configurations, such as memory window of $12 \mu\text{S}$ and noise scale of 0.2. While the influence of drift is still noticeable for longer periods, the reduced drift scale allows for more relaxed hardware constraints.

As the drift scale further reduced to 0.05, there exhibits even broader sets of viable device specifications in long-term scenario. Notably, comparing to drift = 1.0 and 0.5, the accuracy does not degrade significantly across all 3 inference time t : 1 h (Figure 5g), 1 week (Figure 5h), and 1 year (Figure 5i). Under this small drift setting, the LSTM_{HWA} maintains a stable accuracy degradation across all time periods, achieving $<1\%$ degradation even under high noise scales or small memory window. Similar results are obtained when the drift scale is further reduced to 0.005 where additional reduction yields no further improvement.

5 | Results for ResNet-32

Following the HWA training method detailed in General Approach section and Method section on the pretrained FP model, the HWA AIMC ResNet-32 model, denoted as ResNet_{HWA}, achieves iso-accuracy with respect to FP models. Specifically, ResNet_{HWA} achieves iso-accuracy with $\mathcal{A}_*^{1h} \approx 99.0\%$ at 1 h, but with a reduced accuracy at 1 year of $\mathcal{A}_*^{1y} \approx 97.0\%$. These results are consistent with prior work [12], and a detailed summary table is provided in Table S4.

The inference experiments for ResNet_{HWA} follow the same setup as LSTM_{HWA}, detailed in the prior section, where we systematically scan memory window, drift scale and noise scale across 5 inference period $t \in \{1 \text{ s}, 1 \text{ h}, 1 \text{ day}, 1 \text{ week}, 1 \text{ year}\}$. Figure 6 shows the device parameter space at 1 s (Figure 6a), 1 h (Figure 6b), 1 day (Figure 6c), 1 week (Figure 6d), and 1 year (Figure 6e) after weight programing. The surface contour as well as the space enclosed by the surface represent device specifications capable of maintaining $\mathcal{A}_*^t \geq 99.0\%$. We can see a similar trend observed in the LSTM_{HWA} parameter space discussed in the previous section in Figure 3: the contour surface shift downwards as both drift scale and noise scale increase, and the valid parameter space progressively shrinks along these two dimensions as the inference period extends. Compared to LSTM_{HWA} parameter space in Figure 3, the reduction in the ResNet_{HWA} parameter space is substantially greater. The feasible parameter

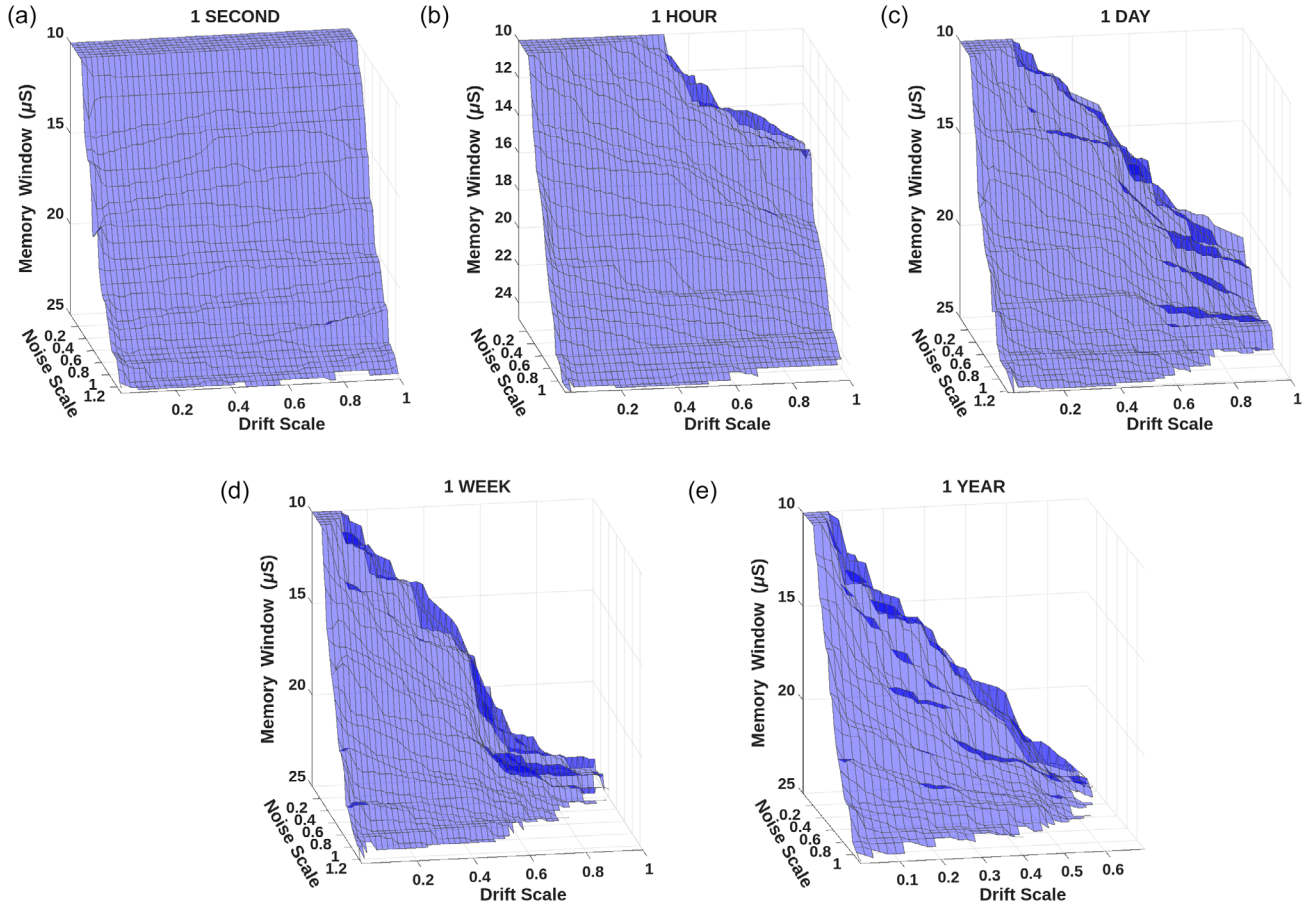


FIGURE 6 | 3D contour plot of PCM device specifications for ResNet_{HWA}. The surface of inference results maintained iso-accuracy $\mathcal{A}_*^t \geq 99\%$ (raw test accuracy of $\approx 93.3\%$, compared to the FP baseline accuracy of 94.1%) with combinations of memory window, noise scale (scale of both programing noise and PCM read noise) and drift scale across 5 different periods t : (a) 1 s, (b) 1 h, (c) 1 Day, (d) 1 Week, (e) 1 Year.

space becomes significantly smaller and more restrictive than that of LSTM_{HWA} at longer time scale. This suggests that while both HWA DNNs exhibit the similar trend, ResNet_{HWA} is more sensitive to device-level imperfections over extended inference durations. Figure S2 shows the feasible parameter space under iso-accuracy thresholds of $A_t^* \geq 99.5\%$ (Figure S2a) and $A_t^* \geq 98\%$ at 1 year (Figure S2b). Consistent with the LSTM case, while the stricter threshold reduces the feasible region and the looser threshold enlarges it, the overall dependence on memory window, drift scale, and noise level remains unchanged. This allows the same design trade-off to be applied across different accuracy requirements. Similarly, to quantitatively evaluate the parameter space, we summarize the noise scale tolerance across memory window and drift scale intervals in Figure 7 at 1 h (Figure 7a), 1 week (Figure 7b), and 1 year (Figure 7c) scenarios. As in the LSTM case, the heatmaps in Figure 7 can be used as lookup tables to determine feasible device specifications for ResNet_{HWA} at different retention durations. For example, at the 1-year inference interval, when the drift scale remains small (e.g., ≤ 0.1), relatively large noise levels can be tolerated with a memory window of 22–25 μS , which supports noise scales up to ≈ 1.0 , whereas

reducing the memory window to 18 μS limits the tolerable noise level to roughly 0.8. As the drift scale increases (e.g., to ≈ 0.4), the allowable noise scale decreases accordingly, with the same memory window of 18 μS now supporting only ≈ 0.5 . Under very high drift (approaching 0.8–1.0), only devices with the largest memory windows (above $\approx 24 \mu\text{S}$) remain viable, and the tolerable noise scale drops below ≈ 0.3 . These lookup tables support the selection of device specifications that balance memory-window, drift, and noise trade-offs to achieve the desired performance targets.

Figure 8 shows the inference result of the ResNet_{HWA} with drift scale of 1.0, 0.5, and 0.05 across 3 inference time t : 1 h, 1 week, and 1 year. Results of 1 s and 1 day are provided in Figure S5. Device characteristic specifications that can sustain iso-accuracy, $A_t^* \geq 99\%$, are highlighted. As shown in Figure 8a (1 h), Figure 8b (1 week), and Figure 8c (1 year), the ResNet_{HWA} model for drift scale = 1 exhibits a trend similar to that of the LSTM_{HWA} model under the same condition: longer inference durations result in a progressively narrower set of device configurations capable of maintaining target accuracy. Meanwhile, as the memory window decreases, the HWA DNN becomes increasingly sensitive to

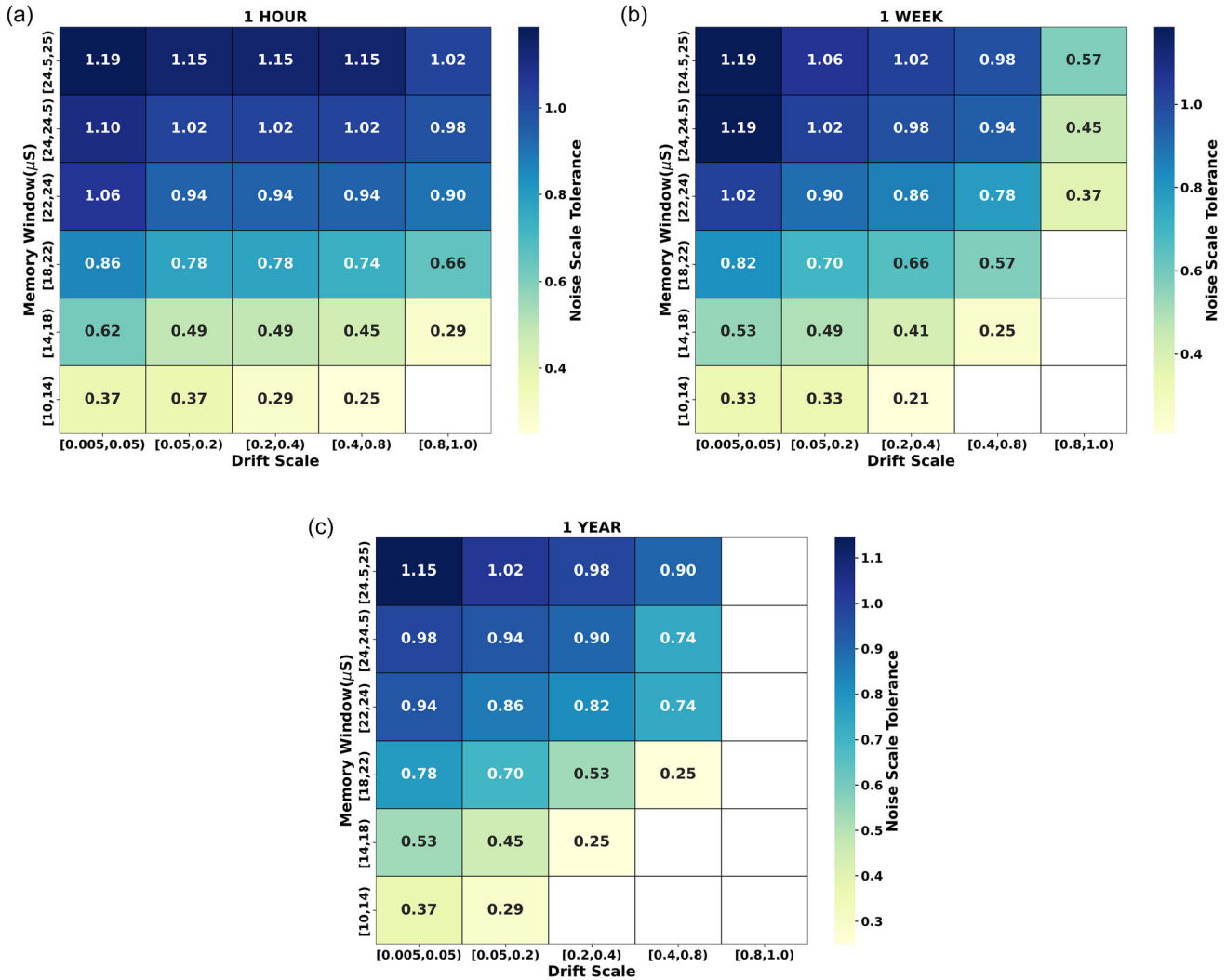


FIGURE 7 | PCM device specifications lookup table for ResNet-32. The numbers in the colored grids show the tolerable noise scales for varying memory windows and resistance drift scales achieving $A_t^* \geq 99\%$ (raw test accuracy of $\approx 93.3\%$ vs. 94.1% for the FP model) at the inference times of: (a) 1 h, (b) 1 Week, (c) 1 Year.

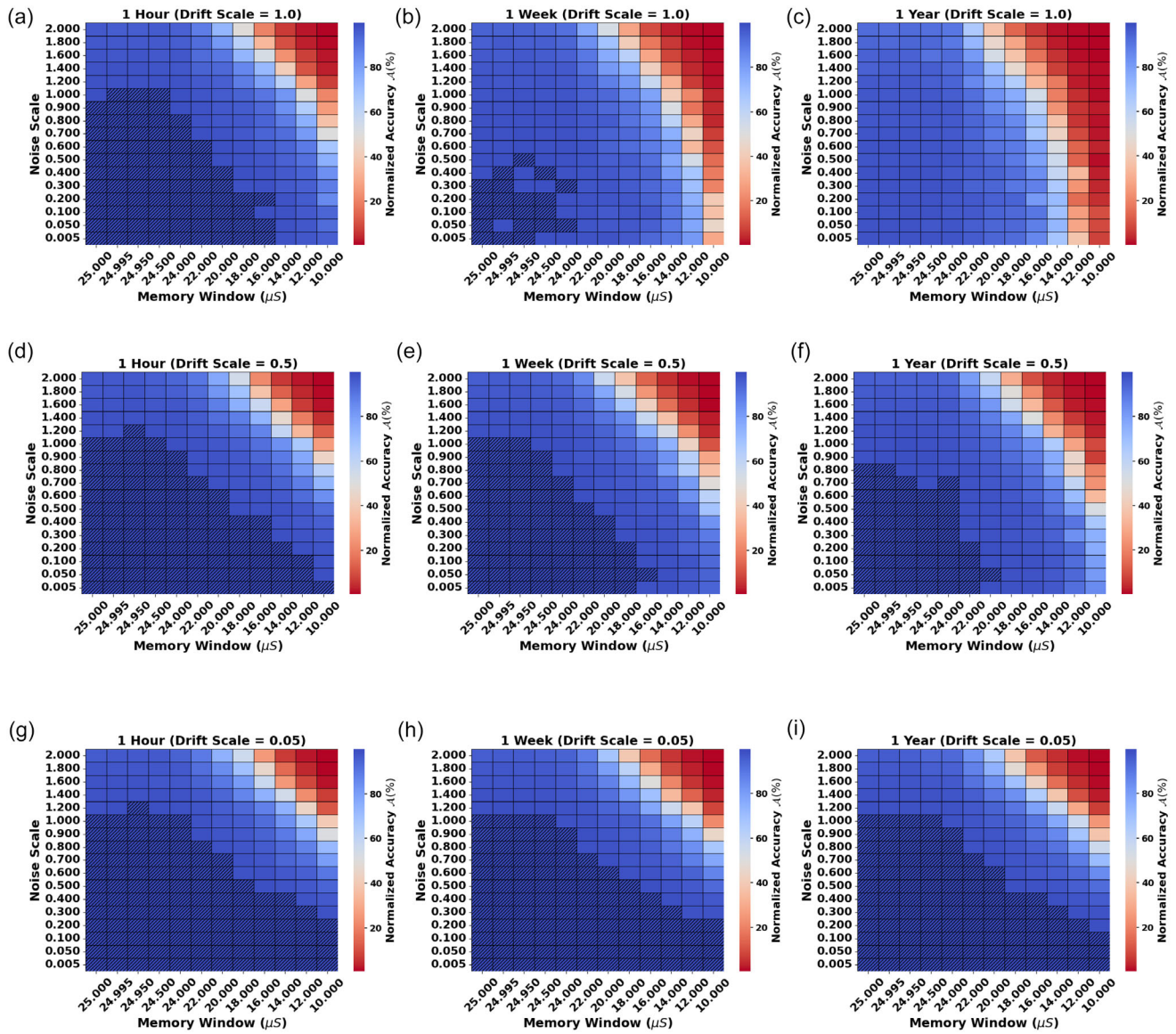


FIGURE 8 | 2D heatmap of AIMC inference accuracy for ResNet_{HWA} for PCM devices with various characteristics. Grids show the normalized accuracy achieved with the varying memory window and noise scale (scale of both programming noise and PCM read noise) across 3 different periods and 3 different drift scales: (a) 1 h (drift scale = 1.0), (b) 1 Week (drift scale = 1.0), (c) 1 Year (drift scale = 1.0), (d) 1 h (drift scale = 0.5), (e) 1 Week (drift scale = 0.5), (f) 1 Year (drift scale = 0.5), (g) 1 h (drift scale = 0.05), (h) 1 Week (drift scale = 0.05), (i) 1 Year (drift scale = 0.05). Grids achieving iso-accuracy $A_t^t \geq 99\%$ are highlighted (raw test accuracy of approximately 93.3%, compared to the FP baseline accuracy of 94.1%).

noise, leading to lower noise tolerance. In contrast to the LSTM_{HWA} model, however, the feasible device specifications for ResNet_{HWA} is notably smaller and more restrictive, and iso-accuracy cannot be achieved at extended inference durations, particularly at $t = 1$ year. As the drift scale reduced to 0.5, we observe wider range of device configurations that meet the iso-accuracy across all 3 inference period: 1 h (Figure 8d), 1 week (Figure 8e), and 1 year (Figure 8f). Feasible configurations now extend toward higher noise scales and smaller memory windows. This relaxation of the specification constraints is particularly evident at longer inference durations. When reducing the drift scale to 0.05, the feasible device specification range further expands. As with the LSTM_{HWA}, the configuration requirements across different inference times become approximately uniform at all evaluated durations including 1 h (Figure 8g), 1 week (Figure 8h), and 1 year (Figure 8i).

6 | Discussion

In this work, we performed systematic investigations to explore options of analog memory device specifications on representative DNN architectures (ResNet-32, LSTM, and BERT-base) by examining the interaction of key device characteristics, providing understanding of their collective influence on the AIMC performance. The results gave a complete selection of viable device characteristics in a multidimensional device specification space, which are capable of maintaining iso-accuracy, i.e., within 1% of the floating point accuracy. These results provide a practical reference for setting target specifications for analog in-memory computing devices and directions for device optimization.

Our experiments demonstrate the quantitative inter-dependencies among drift scale, noise scale, memory window, and inference time. Figures 2, 3, and 6 collectively show that reducing the drift

scale enhances the model's tolerance to noise, thereby improving inference robustness, while increasing the memory window is crucial for preserving accuracy and mitigating the effects of both noise and drift. We quantitatively study the impact of model type and hardware non-idealities, where trade-offs among device parameters can be leveraged to find cost-effective designs among many choices of design parameter combinations for robust AIMC. To provide practical insights for AIMC device specification choices, we summarize some possible design target values for LSTM and ResNet in Figures 4 and 7 for achieving <1% accuracy degradation at 1 h (Figures 4a and 7a), 1 week (Figures 4b and 7b), and 1 year (Figures 4c and 7c) after programming. The figures show that device specifications are not unique and can change with the design or device manufacturing priorities. For example, in some cases, the conductance values $g_{\max}$ and $g_{\min}$ of the analog NVM needs to be determined to satisfy the array leakage current requirement. Once $g_{\max}$ and $g_{\min}$ are determined, noise and the memory window can be optimized to meet one of the many feasible specifications. In some other cases, there is a trade-off between drift, noise, and memory window [26, 27, 39]. The device specification guidelines can help to determine the most easily achievable device characteristics that can achieve iso-accuracy. The sequence of deciding the device specifications can be adjusted based on the designer's and manufacture's preferences.

Furthermore, as part of our simulation experiments on the three neural network models, we observe that different models exhibit varying degrees of robustness to hardware non-idealities, resulting in different feasible device configurations. Specifically, from our results in LSTM and ResNet inference experiments, while both HWA DNNs follow similar trends such as increased noise sensitivity under smaller memory windows and relaxed specification requirements under reduced drift, LSTM consistently shows greater robustness across all inference durations, maintaining iso-accuracy even under more severe noise and drift conditions. In contrast, ResNet requires tighter device specifications to achieve comparable accuracy, especially in the long term. These results demonstrate that the device characteristic specifications for AIMC systems vary depending on the choice of DNN architecture. In Figure S6, we overlay the feasible parameter spaces of the two models under identical inference conditions at 1 h (Figure S6a,d,g), 1 week (Figure S6b,e,h), and 1 year (Figure S6c,f,i). The resulting comparison shows that LSTM

tolerates a wider range of drift and noise than ResNet for the same accuracy target. This result emphasizes that AIMC device specifications are architecture-dependent, and design guidelines should therefore be interpreted relative to the deployed neural network model. In addition, this figure provides a summary of the feasible design space across architectures, and can be used to identify suitable device specifications depending on the deployed DNN architecture.

We created device specification targets versus AIMC performance look-up tables, which can also be used to quickly evaluate the AIMC performance using NVM devices with known specifications. Here, we give some examples on evaluating some recent demonstrations of PCM devices (Table 1). We estimated the AIMC performance of experimentally demonstrated PCM devices by evaluating HWA DNN accuracy $\mathcal{A}_*^t$ under a noise scale of 1. The evaluated devices include the PCM devices with projection liner structure achieving a drift coefficient of less than 0.05 [26, 27], the devices with templated grown homostructure with van der Waals gaps exhibiting a drift coefficient of approximately 0.025 [40], the optimized $\text{Ge}_2\text{Sb}_2\text{Te}_5$ PCM devices [22], high resistivity carbon-based projection layer (CPL) projected PCM [39]. The performance of these devices can be obtained by comparing the characteristics of the device with the device specifications.

We also show that our device specification guidelines can be used to point out the direction and targets for device optimization. For the devices listed in Table 1, we can estimate how much noise needs to be reduced to achieve floating-point accuracy using the specification study. The results are shown as a lookup table in Figure 9 for LSTM (Figure 9a) and ResNet (Figure 9b). Due to the high robustness of the LSTM model, we adopt a stricter criterion of $\mathcal{A}_*^t \geq 99.95\%$ (i.e., $\leq 0.05\%$ accuracy degradation), while for ResNet, the evaluation threshold is maintained at $\mathcal{A}_*^t \geq 99\%$. The noise targets are shown for each device to achieve floating point accuracy at different times after programming, pointing out the directions for optimization.

In summary, achieving robust long-term inference on AIMC systems requires navigating a complex design landscape defined by the interplay of drift, noise, memory window, inference time, etc. Rather than optimizing any single parameter in isolation, a holistic design approach guided by the functional demands of the application and the intended deployment timeframe must be

TABLE 1 | Estimated AIMC accuracy for reported PCM devices.

PCM device technologies	Drift coefficient	Memory window	ResNet-32 accuracy	LSTM accuracy	BERT-base accuracy	Ref.
			$\mathcal{A}_*^t$ (%) (Noise Scale = 1)	$\mathcal{A}_*^t$ (%) (Noise Scale = 1)	$\mathcal{A}_*^t$ (%) (Noise Scale = 1)	
Liner, $\text{Ge}_2\text{Sb}_2\text{Te}_5$	~0.01	from ~0.5 to ~20 μS	~98.92 (1 day), ~98.66 (1 year)	~99.89 (1 day), ~99.81 (1 year)	~96.56 (1 day), ~94.72 (1 year)	[26]
Liner, $\text{Ge}_2\text{Sb}_2\text{Te}_5$	~0.02	from ~1 to ~300 μS	~98.84 (1 day), ~98.70 (1 year)	~99.91 (1 day), ~99.75 (1 year)	~96.26 (1 day), ~95.71 (1 year)	[27]
Homostructure, $\text{Ge}_2\text{Sb}_2\text{Te}_5$	~0.025	from ~1 to ~20 μS	~98.70 (1 day), ~97.46 (1 year)	~99.73 (1 day), ~99.58 (1 year)	~89.08 (1 day), ~85.31 (1 year)	[40]
Optimized $\text{Ge}_2\text{Sb}_2\text{Te}_5$	~0.04	from ~0 to ~75 μS	~98.21 (1 day), ~97.93 (1 year)	~99.73 (1 day), ~99.40 (1 year)	~96.79 (1 day), ~95.80 (1 year)	[22]
CPL projected, doped- $\text{Ge}_2\text{Sb}_2\text{Te}_5$	~0.03	from ~0.1 to ~100 μS	~98.70 (1 day), ~97.98 (1 year)	~99.78 (1 day), ~99.62 (1 year)	~96.83 (1 day), ~95.75 (1 year)	[39]

adopted. Our work provides a quantitative basis for such informed decision-making, bridging device-level properties and system-level robustness in analog in-memory computing.

7 | Method

7.1 | Hardware-Aware (HWA) Training

It has been shown that hardware-aware (HWA) training in software is crucial for improving accuracy for analog inference [4–12]. In our standard HWA training method, we use the same methods as described in [5, 11, 12]. In brief, non-idealities such as input and output quantization, range clippings, and output noise are used in the forward pass, but not in the backward pass, while performing additional stochastic gradient descent (SGD) training to make the model more robust against analog non-idealities. As in [5, 11, 12], we also inject weight noise of PCM characteristics during programing [6], which reversibly modifies the weight matrix for each mini-batch during forward and backward. We consider the learning rates and the strength-scale of the injected weight noise hyper-parameters, and thus scanned those to achieve good training performance with noise (learning rates scanned: from 0.005 to 0.5, weight noise scale scanned: from 0.2 to 5). After hardware-aware-training, converged models are subject to PCM programing noise and drift during inference as described below. To avoid over-fitting in the hyper-parameter search, we trained each parameter setting with 5 different initial conditions and took the model after the same number of training epochs for inference and averaged results. The training results are averaged over 3 trials. For the inference simulations, results are averaged over 25 trials. For LSTM and ResNet experiments, we additionally average across multiple independently trained models, resulting in smoother and more consistent accuracy maps.

For ResNet and BERT, we apply HWA training directly on pre-trained FP models. In contrast, the LSTM model is trained from scratch, including an initial FP training phase. Training details can be obtained from Table S3.

7.2 | Weight Programing

We utilize a previously established and well-characterized weight programing drift model for PCM devices [6, 12, 31], as outlined in the following sections. In this model, each weight is represented by a pair of conductances in the crossbar, with the first conductance programed between reset (0) and a maximum conductance ($\hat{g}_{\max}$) to handle positive weights, and the second conductance programed similarly for negative weights. Only the active conductance is considered in our model, while the non-active conductance is set to the reset state. This approach simplifies the process of computing matrix-vector multiplications (MVM) by using signed conductances.

For conductance initialization, each column, w_i , of the weight matrix is mapped to a corresponding column of target conductances, $\hat{g}_i$. Initially, we assign each affine scale coefficient based on the maximum weight in that column, $\gamma_i = \max |w_{ij}|$. This ensures that each weight is scaled and mapped to a target conductance, $\hat{g}_{ij} = \frac{w_{ij}}{\gamma_i} \hat{g}_{\max}$. Recent approaches [12] involve further optimization of the mapped target conductances and scaling factors, γ_i , using SGD to achieve a better weight-to-conductance conversion, where the factors are further refined by representing them as a product of column-wise γ_i and per-tile k , both of which are learnable. However, our approach does not employ such a learned weight-to-conductance mapping and uses the initialized target conductances without further training.

The programed conductances, g_{ij}^p , deviate from the target values, $\hat{g}_{ij}$, due to programing noise, modeled as Gaussian, resulting in $g_{ij}^p = \hat{g}_{ij} + \sigma^p(\hat{g}_{ij})\xi$, where $\xi \approx \mathcal{N}(0, 1)$. The standard deviation of the programing noise, $\sigma^p(\hat{g})$, depends on the target conductance and is given by:

$$\sigma^p(\hat{g}) = c_0 + \sum_{k=1}^n c_k \frac{\hat{g}^k}{g_{\max}^k} \quad (2)$$

where, $n = 2$, and the coefficients are $c_0 = 0.26348 \mu\text{S}$, $c_1 = 1.9650 \mu\text{S}$, and $c_2 = -1.1731 \mu\text{S}$, derived by fitting to extensive PCM hardware data [31].

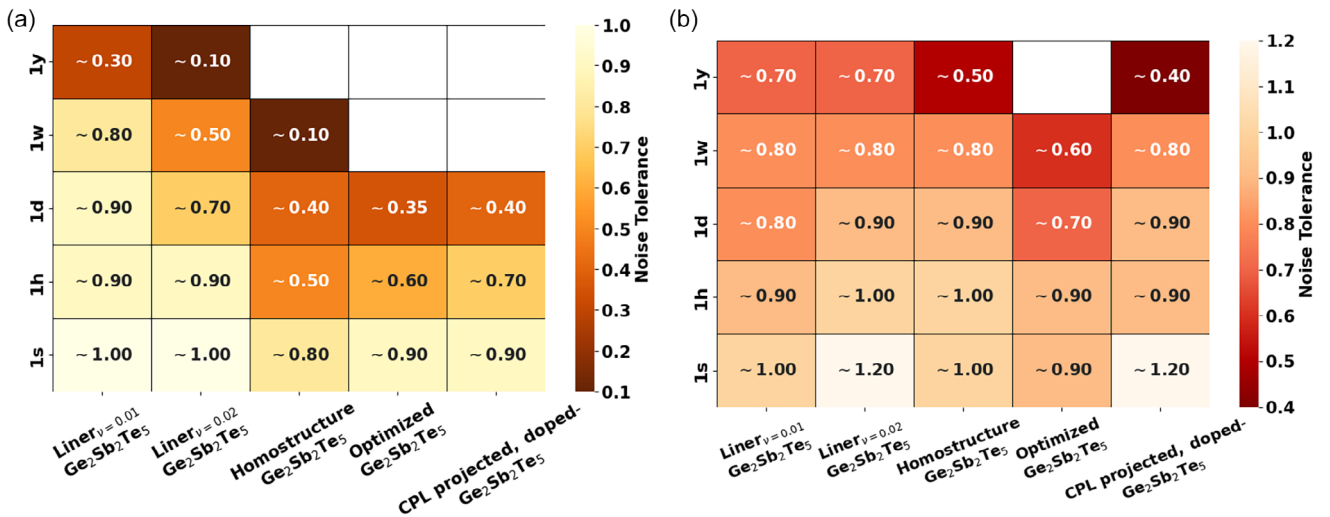


FIGURE 9 | Estimated noise tolerance for LSTM and ResNet Models for reported PCM devices. Each grid cell indicates the maximum tolerable noise scale under which the model maintains acceptable accuracy, based on PCM device specifications (drift coefficient and memory window) from Table 1. (a) LSTM: noise threshold estimated for $\mathcal{A}_*^t \geq 99.95\%$ (i.e., $\leq 0.05\%$ degradation, test perplexity of ≈ 87.1). (b) ResNet: noise threshold estimated for $\mathcal{A}_*^t \geq 99\%$ (i.e., $\leq 1\%$ degradation, raw test accuracy of $\approx 93.3\%$, compared to the FP baseline accuracy of 94.1%).

7.3 | PCM Drift

After a PCM device is programed, it exhibits conductance drift as well as long-term read noise. These behaviors are statistically modeled based on measurements taken from doped-Ge₂Sb₂Te₅ (d-GST) mushroom PCMs [6, 12, 31]. PCM conductance drift follows

$$g^D(t) = g^P \left(\frac{t + t_0}{t_0} \right)^{-\nu} \quad (3)$$

where, $g^D(t)$ is the conductance measured at time t after programing, g^P is the programed conductance, and ν is the drift coefficient. The drift coefficients determined at the beginning for each device are assumed to be normally distributed, that is $\nu_{ij} \sim \mathcal{N}(\mu_\nu(\hat{g}_{ij}), \sigma_\nu(\hat{g}_{ij}))$, where the mean and standard deviation are empirically determined by fitting to PCM hardware data [31]. They are expressed as

$$\begin{aligned} \mu_\nu(\hat{g}) &= \text{clip}_{y_{\min}^\nu}^{y_{\max}^\nu} \left(a \ln \frac{\hat{g}}{g_{\max}} + b \right) \\ \sigma_\nu(\hat{g}) &= \text{clip}_{y_{\min}^\sigma}^{y_{\max}^\sigma} \left(c \ln \frac{\hat{g}}{g_{\max}} + d \right) \end{aligned} \quad (4)$$

where, $y_{\min}^\nu = 0.049$, $y_{\max}^\nu = 0.1$, $a = -0.0155$, $b = 0.0244$ (for $\mu_\nu(\hat{g})$), $y_{\min}^\sigma = 0.008$, $y_{\max}^\sigma = 0.045$, $c = -0.0125$, and $d = -0.0059$ (for $\sigma_\nu(\hat{g})$).

7.4 | Long Term Read Noise

Similar to many AIMC devices, PCM is also known to exhibit low-frequency noise, including $1/f$ noise and random telegraph noise, which affect the effective conductance values used in computations. Unlike short-term read noise, which causes weight read fluctuations within the time scale of a single MVM, this read noise is resampled only once at each inference time. In our approach, we apply the noise model established in [12, 31], where the standard deviation of read noise at time t as

$$\sigma_{\text{read}}(t) = \hat{g} Q_s(\hat{g}) \sqrt{\ln \left(\frac{t + t_{\text{read}}}{2t_{\text{read}}} \right)} \quad (5)$$

$Q_s(\hat{g})$ is determined to be

$$Q_s(\hat{g}) = \text{clip}_0^a \left(b \left(\frac{\hat{g}}{g_{\max}} \right)^c \right) \quad (6)$$

where, $a = 0.2$, $b = 0.0088$, and $c = -0.65$.

Author Contributions

Zhenyu Wu: data curation (lead), formal analysis (lead), investigation (lead), methodology (equal), software (equal), validation (lead), visualization (lead), writing – original draft (lead), writing – review and editing (lead), **Xin Su:** data curation (equal), formal analysis (equal), investigation (equal), software (equal), validation (equal), visualization (equal), writing – original draft (equal), writing – review and editing (equal), **Malte J. Rasch:** formal analysis (equal), investigation (equal), methodology (equal), software (lead), validation (lead), writing – review and editing (equal), **Ning Li:** conceptualization (lead), formal analysis (lead), funding acquisition (lead), investigation (equal), methodology (lead), project administration (lead), resources (lead), supervision (lead),

validation (equal), writing – original draft (lead), writing – review and editing (lead).

Funding

This work was supported by Penn State University.

Conflicts of Interest

The authors declare no conflicts of interest.

Data Availability Statement

The data that support the findings of this study are available from the corresponding author upon reasonable request.

References

1. G. W. Burr, R. M. Shelby, A. Sebastian, et al., “Neuromorphic Computing Using Non-Volatile Memory,” *Advanced Physics X* 2, no. 1 (2017): 89–124.
2. A. Sebastian, M. Le Gallo, R. Khaddam-Aljameh, and E. Eleftheriou, “Memory Devices and Applications for in-Memory Computing,” *Nature Nanotechnology* 15 (2020): 529–544.
3. D. Kudithipudi, C. Schuman, C. M. Vineyard, et al., “Neuromorphic Computing at Scale,” *Nature* 637, no. 8047 (2025): 801–812, <https://doi.org/10.1038/s41586-024-08253-8>.
4. M. J. Rasch, T. Gokmen, M. Rigotti, and W. Haensch, “RAPA-ConvNets: Modified Convolutional Networks for Accelerated Training on Architectures with Analog Arrays,” *Frontiers in Neuroscience* 13 (2019): 753.
5. C. Mackin, M. J. Rasch, A. Chen, et al., “Optimised Weight Programming for Analogue Memory-Based Deep Neural Networks,” *Nature Communications* 13, no. 1 (2022): 3765.
6. V. Joshi, M. Le Gallo, S. Haefeli, et al., “Accurate Deep Neural Network Inference Using Computational Phase-Change Memory,” *Nature Communications* 11, no. 1 (2020): 2473, <https://doi.org/10.1038/s41467-020-16108-9>.
7. H. Tsai, S. Ambrogio, C. Mackin, et al., “Inference of Long-Short Term Memory Networks at Software-Equivalent Accuracy Using 2.5M Analog Phase Change Memory Devices,” in *2019 Symposium on VLSI Technology* (IEEE, 2019), T82–T83, <https://doi.org/10.23919/VLSIT.2019.8776519>.
8. X. Yang, C. Wu, M. Li, and Y. Chen, “Tolerating Noise Effects in Processing-in-Memory Systems for Neural Networks: A Hardware-Software Codeign Perspective,” *Advanced Intelligent Systems* 4, no. 6 (2022): 2200029.
9. T. Gokmen, M. J. Rasch, and W. Haensch, “The Marriage of Training and Inference for Scaled Deep Learning Analog Hardware,” in *2019 IEEE International Electron Devices Meeting (IEDM)* (IEEE, 2019), 22–23.
10. S. Kariyappa, H. Tsai, K. Spoon, et al., “Noise-Resilient DNN: Tolerating Noise in PCM-Based AI Accelerators via Noise-Aware Training,” *IEEE Transactions on Electron Devices* 68, no. 1 (2021): 1–7.
11. K. Spoon, H. Tsai, A. Chen, et al., “Toward Software-Equivalent Accuracy on Transformer-Based Deep Neural Networks with Analog Memory Devices,” *Frontiers in Computational Neuroscience* 15 (2021): 1–9.
12. M. J. Rasch, C. Mackin, M. Le Gallo, et al., “Hardware-Aware Training for Large-Scale and Diverse Deep Learning Inference Workloads Using in-Memory Computing-Based Accelerators,” *Nature Communications* 14, no. 1 (2023): 5282, <https://doi.org/10.1038/s41467-023-40770-4>.
13. S. Ambrogio, M. Gallot, K. Spoon, et al., “Reducing the Impact of Phase-Change Memory Conductance Drift on the Inference of Large-

- Scale Hardware Neural networks,” in *2019 IEEE International Electron Devices Meeting (IEDM)* (IEEE, 2019), 1–4.
14. G. W. Burr, M. J. Brightsky, A. Sebastian, et al., “Recent Progress in Phase-Change Memory Technology,” *IEEE Journal on Emerging and Selected Topics in Circuits and Systems* 6, no. 2 (2016): 146–162.
 15. M. Le Gallo and A. Sebastian, “An Overview of Phase-Change Memory Device Physics,” *Journal of Physics D: Applied Physics* 53, no. 21 (2020): 213002.
 16. H. Tsai, S. Ambrogio, P. Narayanan, R. M. Shelby, and G. W. Burr, “Recent Progress in Analog Memory-Based Accelerators for Deep Learning,” *Journal of Physics D: Applied Physics* 51, no. 28 (2018): 283001.
 17. X. Su, Z. Guo, G. Zhang, H. Tsai, and N. Li, “Progress and Challenges of Phase Change Memory for in-Memory Computing,” *Current Opinion in Solid State and Materials Science* 37 (2025): 101225, <https://doi.org/10.1016/j.cossms.2025.101225>.
 18. J. McDonald, et al., “Great Power, Great Responsibility: Recommendations for Reducing Energy for Training Language Models,” in Findings of the Association for Computational Linguistics: NAACL 2022 (Association for Computational Linguistics, 2022), 1962–1970. arXiv:2205.09646 [cs]. <https://doi.org/10.18653/v1/2022.findings-naacl.151>.
 19. J.-W. Jang, S. Park, G. W. Burr, H. Hwang, and Y.-H. Jeong, “Optimization of Conductance Change in Pr1-xCaxMnO3-Based Synaptic Devices for Neuromorphic Systems,” *IEEE Electron Device Letters* 36, no. 5 (2015): 457–459.
 20. W.-H. Chen, C. Dou, K.-X. Li, et al., “CMOS-Integrated Memristive Non-Volatile Computing-in-Memory for AI Edge Processors,” *Nature Electronics* 2 (2019): 420–428.
 21. P. Deaville, B. Zhang, and N. Verma, “A 22nm 128-kb MRAM Row/Column-Parallel in-Memory Computing Macro with Memory-Resistance Boosting and Multi-Column ADC Read-Out,” in *2022 IEEE Symposium on VLSI Technology and Circuits (VLSI Technology and Circuits)* (IEEE, 2022), 268–269.
 22. I. Boybat, T. Boesch, M. Allegra, et al., “Heterogeneous Embedded Neural Processing Units Utilizing PCM-Based Analog In-Memory Computing,” in *2024 IEEE International Electron Devices Meeting (IEDM)* (IEEE, 2024), 1–4, <https://doi.org/10.1109/IEDM50854.2024.10873479>.
 23. S. R. Nandakumar, I. Boybat, J.-P. Han, et al., “Precision of Synaptic Weights Programmed in Phase-Change Memory Devices for Deep Learning Inference,” in *2020 IEEE International Electron Devices Meeting (IEDM)* (IEEE, 2020), 29.4.1–29.4.4, <https://doi.org/10.1109/IEDM13553.2020.9371990>.
 24. L. Chen, J. Li, Y. Chen, et al., “Accelerator-Friendly Neural-Network Training: Learning Variations and Defects in RRAM Crossbar,” Design, Automation & Test in Europe Conference & Exhibition (DATE), (European Design and Automation Association, 2017), 19–24.
 25. T. M. Philip, K. W. Brew, N. Li, et al., “Design of Projected Phase-Change Memory Mushroom Cells for Low-Resistance Drift,” *MRS Bulletin* 48 (2022): 228–236, <https://doi.org/10.1557/s43577-022-00391-6>.
 26. R. L. Bruce, S. G. Sarwat, I. Boybat, et al., “Mushroom-Type Phase Change Memory with Projection Liner: An Array-Level Demonstration of Conductance Drift and Noise Mitigation,” in *2021 IEEE International Reliability Physics Symposium (IRPS)* (IEEE, 2021), 1–6, <https://doi.org/10.1109/IRPS46558.2021.9405191>.
 27. N. Li, C. Mackin, A. Chen, et al., “Optimization of Projected Phase Change Memory for Analog In-Memory Computing Inference,” *Advanced Electronic Materials* 9, no. 6 (2023): 2201190, <https://doi.org/10.1002/aelm.202201190>.
 28. K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning For Image Reconvition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (IEEE, 2016), 770–778, <https://doi.org/10.1109/CVPR.2016.90>.
 29. S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation* 9, no. 8 (1997): 1735–1780, <https://doi.org/10.1162/neco.1997.9.8.1735>.
 30. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-Training of Deep Bidirectional Transformers for Language Understanding,” arXiv:1810.04805 [cs], 2019.
 31. S. R. Nandakumar, I. Boybat, V. Joshi, et al., “Phase-Change Memory Models for Deep Learning Training and Inference,” in *2019 26th IEEE International Conference on Electronics, Circuits and Systems (ICECS)* (IEEE, 2019), 727–730, <https://doi.org/10.1109/ICECS46596.2019.8964852>.
 32. M. J. Rasch, D. Moreda, T. Gokmen, et al., “A Flexible and Fast PyTorch Toolkit for Simulating Training and Inference on Analog Crossbar Arrays,” in *2021 IEEE International Conference on Artificial Intelligence Circuits and Systems (AICAS)* (IEEE, 2021), 1–4.
 33. M. J. Rasch, T. Gokmen, and W. Haensch, “Training Large-Scale Artificial Neural Networks on Simulated Resistive Crossbar Arrays,” *IEEE Design & Test* 37, no. 6 (2019): 19–29.
 34. M. Le Gallo, C. Lammie, J. Büchel, et al., “Using the IBM Analog in-Memory Hardware Acceleration Kit for Neural Network Training and Inference,” *APL Machine Learning* 1, no. 4 (2023): 041102, <https://doi.org/10.1063/5.0168089>.
 35. A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images* (Tech. rep., University of Toronto, 2009).
 36. P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang, “SQuAD: 100,000+ Questions for Machine Comprehension of Text,” arXiv:1606.05250 [cs], 2016.
 37. A. Wang, A. Singh, J. Michael, F. Hill, and S. R. Bowman, “GLUE: A Multi-Task Benchmark and Analysis Platform for Natural Language Understanding,” in *International Conference on Learning Representations (ICLR)* (OpenReview.net, 2019).
 38. A. Taylor, M. Marcus, and B. Santorini, “The Penn Treebank: An Overview,” in *Treebanks: Building and Using Parsed Corpora*, ed. A. Abeillé (Kluwer Academic Publishers, 2003), 5–22.
 39. G. S. Syed, K. Brew, A. Vasilopoulos, et al., “In-Memory Compute Chips with Carbon-Based Projected Phase-Change Memory Devices,” in *2023 International Electron Devices Meeting (IEDM)* (IEEE, 2023), 1–4, <https://doi.org/10.1109/IEDM45741.2023.10413792>.
 40. G. M. Cohen, A. Majumdar, C-Wei Cheng, et al., “Low RESET Current Mushroom-Cell Phase-Change Memory Using Fiber-Textured Homostructure GeSbTe on Highly Oriented Seed Layer,” *Physica Status Solidi (RRL) – Rapid Research Letters* 18 (2024): 2300426, <https://doi.org/10.1002/pssr.202300426>.

Supporting Information

Additional supporting information can be found online in the Supporting Information section. **Supporting Fig. S1:** 3D contour plot of PCM device specifications for LSTM_{HWA}. The surface of inference results maintained accuracy criterions of: **a.** $\mathcal{A}_*^t \geq 99.5\%$ (test perplexity of approximately 87.1), **b.** $\mathcal{A}_*^t \geq 98\%$ (test perplexity of approximately 89.4), with combinations of memory window, noise scale (scale of both programming noise and PCM read noise) and drift scale at 1 year. **Supporting Fig. S2:** 3D contour plot of PCM device specifications for ResNet_{HWA}. The surface of inference results maintained accuracy criterions of: **a.** $\mathcal{A}_*^t \geq 99.5\%$ (raw test accuracy of approximately 93.7%), **b.** $\mathcal{A}_*^t \geq 98\%$ (raw test accuracy of approximately 92.4%), with combinations of memory window, noise scale (scale of both programming noise and PCM read noise) and drift scale at 1 year. **Supporting Fig. S3:** Gradient field of accuracy sensitivity in the noise-memory window parameter space of LSTM_{HWA} on PCM. The color map represents normalized accuracy $\mathcal{A}_n$, with contour lines at 95% (perplexity of approximately 95.1), 98% (perplexity of approximately 89.4), and 99% (perplexity of approximately 87.8), across 3 different periods and 3 different drift scales: **a.** 1 Hour (drift scale=1.0), **b.** 1 Week (drift scale=1.0), **c.** 1 Year (drift scale=1.0), **d.** 1 Hour (drift scale=0.5), **e.** 1 Week (drift scale=0.5).

scale=0.5), **f.** 1 Year (drift scale=0.5), **g.** 1 Hour (drift scale=0.05), **h.** 1 Week (drift scale=0.05), **i.** 1 Year (drift scale=0.05). Arrows show the gradient direction $\nabla \mathcal{A}_t$, with length proportional to gradient magnitude. Gaussian smoothing ($\sigma = 1.0$) applied to reduce accuracy fluctuation.

Supporting Fig. S4: AIMC inference of LSTM_{HWA} on PCM. Grids show the normalized accuracy achieved with the varying memory window and noise scale (scale of both programming noise and PCM read noise) across 2 different periods and 3 different drift scales: **a.** 1 (drift scale=1.0), **b.** 1 Day (drift scale=1.0), **c.** 1 Second (drift scale=0.5), **d.** 1 Day (drift scale=0.5), **e.** 1 Second (drift scale=0.05), **f.** 1 Day (drift scale=0.05). Grids achieving iso-accuracy $\mathcal{A}_*^t \geq 99\%$ are highlighted (test perplexity of approximately 87.8).

Supporting Fig. S5: AIMC inference of ResNet_{HWA} on PCM. Grids show the normalized accuracy achieved with the varying memory window and noise scale (scale of both programming noise and PCM read noise) across 2 different periods and 3 different drift scales: **a.** 1 Second (drift scale=1.0), **b.** 1 Day (drift scale=1.0), **c.** 1 Second (drift scale=0.5), **d.** 1 Day (drift scale=0.5), **e.** 1 Second (drift scale=0.05), **f.** 1 Day (drift scale=0.05). Grids achieving iso-accuracy $\mathcal{A}_*^t \geq 99\%$ are highlighted (raw test accuracy of approximately 93.3%, compared to the FP baseline accuracy of 94.1%).

Supporting Fig. S6: AIMC inference of ResNet_{HWA} and LSTM_{HWA} on PCM. Grids show the normalized accuracy achieved with the varying memory window and noise scale (scale of both programming noise and PCM read noise) across 3 different periods and 3 different drift scales: **a.** 1 Hour (drift scale=1.0), **b.** 1 Week (drift scale=1.0), **c.** 1 Year (drift scale=1.0), **d.** 1 Hour (drift scale=0.5), **e.** 1 Week (drift scale=0.5), **f.** 1 Year (drift scale=0.5), **g.** 1 Hour (drift scale=0.05), **h.** 1 Week (drift scale=0.05), **i.** 1 Year (drift scale=0.05). Grids achieving iso-accuracy $\mathcal{A}_*^t \geq 99\%$ are highlighted (corresponding to test perplexity of 87.8 for LSTM_{HWA} and test accuracy of 93.3% for ResNet_{HWA}).

Supporting Table S1: PCM Device Specifications. **Supporting Table S2:** Base parameters of the default AIHWKIT PCM noise model. **Supporting Table S3:** Hyper-parameters for training LSTM, ResNet, and BERT. **Supporting Table S4:** AIMC Test Error of LSTM_{HWA} and ResNet_{HWA}.